Série d'exercices : PHP procédural

echo – chaînes – define – include – die – GET / POST – conversion – tableaux – dates – boutons radio – MySQLi

4ème SI – Lycée Rafèha – Enseignant : Aymen Essouyah

CORRIGÉ

Sommaire

Partie 1 – echo, variables et chaînes (' et ") 1

Partie 2 – define, include et die 4

Partie 3 – GET et POST 6

Partie 4 – Conversion, conditions et boucles 9

Partie 5 – Tableaux et tableaux associatifs 12

Partie 6 – Les dates 15

Partie 7 – Formulaires avec boutons radio 21

Partie 8 – MySQLi procédural (base Dar Zitouna) 26

À propos du corrigé
- Chaque solution a été **exécutée** (PHP 8.3, MariaDB 10.11) : les cadres à bord noir montrent l'affichage réel obtenu avec les données indiquées.
- Pour les exercices sur les dates, la date du jour était le 14/11/2026. Pour les formulaires, le cadre montre le **texte du résultat** (le formulaire HTML n'est pas reproduit).
- D'autres solutions correctes existent : l'essentiel est de respecter les consignes et le jeu d'essai.

Partie 1 – echo, variables et chaînes (' et ")

Exercice 1.1 – Prédire l'affichage (guillemets simples et doubles)

Écrire **exactement** ce que le navigateur affiche (une ligne par `<br>`).

```
<?php
$nom = "Rim";
$age = 17;
echo 'Nom : $nom<br>';
echo "Nom : $nom<br>";
echo "Dans un an : " . ($age + 1) . " ans<br>";
echo "Prix : \$5 ou 5 DT<br>";
echo 'L\'été<br>';
echo $nom . $age . "<br>";
echo $nom, $age + 3;
?>
```

Solution :

Affichage :

```
Nom : $nom
Nom : Rim
Dans un an : 18 ans
Prix : $5 ou 5 DT
L'été
Rim17
Rim20
```

→ Ligne 1 : guillemets **simples** → `$nom` n'est pas remplacé.

→ Ligne 4 : `\$` affiche un vrai `$`. Ligne 5 : `\'` affiche l'apostrophe.

→ Dernière ligne : la virgule sépare deux arguments de `echo` ; `$age + 3` vaut 20 (calcul, pas concaténation) ; rien n'est ajouté entre `Rim` et `20`.

Exercice 1.2 – Corriger les erreurs

Ce script contient **cinq erreurs** (syntaxe, guillemets, opérateur, variable). Les repérer, les expliquer, puis réécrire le script corrigé. Il doit afficher : le message de bienvenue, l'âge, un champ de saisie pré-rempli et le total.

```
<?php
nom = "Sami"
$age = "17";
echo "Bonjour $nom<br>";
echo "Vous avez " + $age + " ans<br>";
echo "<input type="text" name="nom" value="$nom"><br>";
echo 'Total : ' . $prix . ' DT';
?>
```

Solution :

Fichier : corrige.php

```
<?php
$nom = "Sami";
$age = "17";
$prix = 25;
echo "Bonjour $nom<br>";
echo "Vous avez " . $age . " ans<br>";
echo "<input type='text' name='nom' value='$nom'><br>";
echo 'Total : ' . $prix . ' DT';
?>
```

Affichage obtenu

```
Bonjour Sami<br>
Vous avez 17 ans<br>
<input type='text' name='nom' value='Sami'>
<br>
Total : 25 DT
```

→ 1) `nom` sans `$`. 2) `;` oublié après `"Sami"`. 3) `+` au lieu de `.` pour concaténer (`+` est arithmétique).

→ 4) Guillemets doubles à l'intérieur de guillemets doubles : utiliser `'` pour les attributs HTML (ou `\"`).

→ 5) `$prix` n'a jamais reçu de valeur : « Undefined variable $prix ».

Exercice 1.3 – Écrire du HTML avec echo

Avec `$nom = "Sami";` et `$prix = 12.5;`, écrire **deux versions** de l'instruction `echo` qui produisent exactement la ligne HTML suivante : version A avec des guillemets simples pour PHP, version B avec des guillemets doubles pour PHP.

```
<a href="fiche.php?nom=Sami&prix=12.5" class="lien">Sami</a>
```

Solution :

Fichier : html.php

```
<?php
$nom = "Sami";
$prix = 12.5;
echo '<a href="fiche.php?nom=' . $nom . '&prix=' . $prix . '" class="lien">' . $nom . '</a>';
echo "\n";
echo "<a href=\"fiche.php?nom=$nom&prix=$prix\" class=\"lien\">$nom</a>";
?>
```

Affichage obtenu

```
<a href="fiche.php?nom=Sami&prix=12.5" class="lien">Sami</a>
<a href="fiche.php?nom=Sami&prix=12.5" class="lien">Sami</a>
```

→ Version A : PHP entre `'...'`, HTML avec `"..."`, variables collées avec l'opérateur `.` (point).

→ Version B : PHP entre `"..."` : les variables sont remplacées directement, mais les " du HTML doivent être échappés (`\"`). On aurait aussi pu écrire les attributs avec des apostrophes.

Exercice 1.4 – Fonctions sur les chaînes

Avec `$s = " dar zitouna ";`, écrire le code qui affiche, **chacun sur une ligne** :

a) la chaîne sans les espaces de début et de fin, entre crochets : `[dar zitouna]`

b) son nombre de caractères (après suppression des espaces)

c) chaque mot avec une majuscule, puis tout en majuscules

d) la chaîne où chaque `o` est remplacé par `0`

e) ses 3 premières lettres, puis la position du mot `zit`

f) le 2e mot (avec `strpos` et `substr`) et le nombre de mots (avec `substr_count`)

Solution :

Fichier : chaines.php

```php
<?php
$s = "  dar zitouna  ";
$t = trim($s);
echo "[" . $t . "]<br>";
echo strlen($t) . "<br>";
echo ucwords($t) . " / " . strtoupper($t) . "<br>";
echo str_replace("o", "0", $t) . "<br>";
echo substr($t, 0, 3) . " / " . strpos($t, "zit") . "<br>";
$p = strpos($t, " ");
echo substr($t, $p + 1) . " / " . (substr_count($t, " ") + 1);
?>
```

Affichage obtenu

```
[dar zitouna]
11
Dar Zitouna / DAR ZITOUNA
dar zit0una
dar / 4
zitouna / 2
```

→ `trim` d'abord, sinon `strlen` compterait aussi les espaces (15 au lieu de 11) et `strpos` serait décalé de 2.

→ Le 2e mot commence juste après le premier espace : `strpos($t, " ")` donne sa position (3), `substr($t, $p + 1)` extrait la fin. Nombre de mots = nombre d'espaces + 1 (mots séparés par un seul espace).

Partie 2 – define, include et die

Exercice 2.1 – Constantes et TVA

Écrire `tva.php` : définir avec `define` les constantes `TVA` (0.19), `DEVISE` ("DT") et `RESTAURANT` ("Dar Zitouna"). À partir du tableau associatif `$plats = ["Couscous" => 22, "Poisson" => 28, "Lablabi" => 9];` (prix hors taxe), afficher le nom du restaurant puis, pour chaque plat (parcourir les clés avec `array_keys` et une boucle `for`), le prix HT, la TVA et le prix TTC avec 2 décimales. Pourquoi `echo "TVA";` n'affiche-t-il pas 0.19 ?

Solution :

Fichier : tva.php

```
<?php
define("TVA", 0.19);
define("DEVISE", "DT");
define("RESTAURANT", "Dar Zitouna");
$plats = ["Couscous" => 22, "Poisson" => 28, "Lablabi" => 9];
$noms = array_keys($plats);
echo "<h3>" . RESTAURANT . "</h3>";
for ($i = 0; $i < count($noms); $i++) {
    $nom = $noms[$i];
    $ht = $plats[$nom];
    $tva = $ht * TVA;
    echo $nom . " : HT " . number_format($ht, 2) . " " . DEVISE;
    echo ", TVA " . number_format($tva, 2);
    echo ", TTC " . number_format($ht + $tva, 2) . " " . DEVISE . "<br>";
}
?>
```

Affichage obtenu

```
Dar Zitouna
Couscous : HT 22.00 DT, TVA 4.18, TTC 26.18 DT
Poisson : HT 28.00 DT, TVA 5.32, TTC 33.32 DT
Lablabi : HT 9.00 DT, TVA 1.71, TTC 10.71 DT
```

→ Une constante s'écrit **sans** `$` et ne s'interprète pas dans une chaîne : `"TVA"` est simplement le mot TVA ; il faut écrire `echo TVA;` ou `"..." . TVA`.

→ `array_keys($plats)` renvoie le tableau indexé des clés (`Couscous`, `Poisson`, `Lablabi`) : on peut alors le parcourir avec `for` et retrouver chaque prix par `$plats[$nom]`.

Exercice 2.2 – Organiser un site avec include et require

Créer quatre fichiers : `config.php` (constantes `RESTAURANT` et `ANNEE`, valant 2026), `entete.php` (titre du restaurant et menu avec deux liens : Accueil, Contact), `pied.php` (copyright avec l'année) et `accueil.php` qui les assemble. Justifier le choix de `require` ou `include` pour chaque fichier.

Solution :

Fichier : config.php

```
<?php
define("RESTAURANT", "Dar Zitouna");
define("ANNEE", 2026);
?>
```

Fichier : entete.php

```
<h1><?= RESTAURANT ?></h1>
<a href="accueil.php">Accueil</a> | <a href="contact.php">Contact</a>
<hr>
```

Fichier : pied.php

```
<hr>
<p>&copy; <?= ANNEE ?> <?= RESTAURANT ?></p>
```

Fichier : accueil.php

```
<?php
require "config.php";
?>
<html><body>
<?php include "entete.php"; ?>
<p>Bienvenue dans notre restaurant.</p>
<?php include "pied.php"; ?>
</body></html>
```

Affichage obtenu

```
<html><body>
<h1>Dar Zitouna</h1>
<a href="accueil.php">Accueil</a> | <a href="contact.php">Contact</a>
<hr>
<p>Bienvenue dans notre restaurant.</p>
<hr>
<p>&copy; 2026 Dar Zitouna</p>
</body></html>
```

→ `require` pour `config.php` : sans constantes la page n'a pas de sens (erreur fatale si absent). `include` pour l'en-tête et le pied : éléments d'affichage, la page peut continuer sans eux.

→ Si `config.php` risque d'être inclus plusieurs fois : `require_once`.

Exercice 2.3 – Arrêter le script avec die

Écrire `fiche.php` qui reçoit un numéro `num` dans l'adresse et utilise le tableau `$serveurs = [1 => "Sami", 2 => "Rim", 3 => "Karim", 4 => "Nour"];`. Utiliser `die` pour : numéro absent (« Numéro manquant »), numéro non numérique (« Numéro invalide »), numéro absent du tableau (« Serveur inconnu »). Sinon afficher « Serveur n°3 : Karim ».

Solution :

Fichier : fiche.php

```
<?php
$serveurs = [1 => "Sami", 2 => "Rim", 3 => "Karim", 4 => "Nour"];
if (!isset($_GET["num"])) {
    die("Numéro manquant");
}
$num = $_GET["num"];
if (!is_numeric($num)) {
    die("Numéro invalide");
}
if (!isset($serveurs[$num])) {
    die("Serveur inconnu");
}
echo "Serveur n°$num : " . $serveurs[$num];
?>
```

Cas « fiche.php?num=3 » – données envoyées : num = 3

```
Serveur n°3 : Karim
```

Cas « fiche.php »

```
Numéro manquant
```

Cas « fiche.php?num=abc » – données envoyées : num = abc

```
Numéro invalide
```

Cas « fiche.php?num=9 » – données envoyées : num = 9

```
Serveur inconnu
```

→ L'ordre des tests compte : `isset` avant de lire `$_GET["num"]` (sinon « Undefined array key »), puis `is_numeric`, puis l'existence dans le tableau.

Partie 3 – GET et POST

Exercice 3.1 – Vrai ou faux ?

Répondre par **V** ou **F** et corriger les affirmations fausses.

a) `$_POST["nom"]` lit le champ dont l'attribut `id` vaut `nom`.

b) Avec `method="get"`, les données du formulaire apparaissent dans l'adresse de la page.

c) Un formulaire de connexion doit utiliser la méthode GET.

d) Un bouton radio non coché est envoyé avec une valeur vide.

e) Les valeurs de `$_POST` sont toujours de type chaîne.

f) `isset($_POST["valider"])` permet de savoir si le formulaire a été soumis (le bouton a un `name`).

g) On peut lire `$_GET["x"]` sans le tester : au pire la valeur est vide.

h) `include "a.php";` arrête le script si `a.php` est absent.

Solution :

a) **F** : c'est l'attribut `name` qui donne la clé.

b) **V**.

c) **F** : POST (les données ne doivent pas apparaître dans l'adresse ni l'historique).

d) **F** : il n'est **pas envoyé du tout** ; la clé n'existe pas dans `$_POST` → `isset`.

e) **V** : même un champ numérique ou une date arrive sous forme de chaîne.

f) **V**.

g) **F** : une clé absente provoque « Warning: Undefined array key » ; toujours tester avec `isset`.

h) **F** : `include` donne un avertissement et le script continue ; c'est `require` qui s'arrête (erreur fatale).

Exercice 3.2 – Inscription d'un serveur (POST)

Écrire `inscription.php` (formulaire et traitement dans le même fichier, méthode POST) avec : `nom` et `prenom` (texte), `salle` (liste déroulante : Terrasse, Salon) et le bouton `inscrire`. Après envoi : si un des deux champs est vide (utiliser `trim` et `empty`), afficher « Nom et prénom obligatoires. » ; sinon afficher « Bienvenue Prénom NOM, vous travaillerez en salle X. » (prénom avec majuscule initiale, nom en majuscules).

Solution :

Fichier : inscription.php

```
<?php
if (isset($_POST["inscrire"])) {
    $nom = trim($_POST["nom"]);
    $prenom = trim($_POST["prenom"]);
    $salle = $_POST["salle"];
    echo "<div id='resultat'>";
    if (empty($nom) || empty($prenom)) {
        echo "<p>Nom et prénom obligatoires.</p>";
    } else {
        echo "<p>Bienvenue " . ucfirst(strtolower($prenom)) . " " . strtoupper($nom);
        echo ", vous travaillerez en salle $salle.</p>";
    }
    echo "</div>";
}
?>
<form method="post">
  Nom : <input type="text" name="nom"><br>
  Prénom : <input type="text" name="prenom"><br>
  Salle : <select name="salle">
    <option value="Terrasse">Terrasse</option>
    <option value="Salon">Salon</option>
  </select><br>
  <input type="submit" name="inscrire" value="S'inscrire">
</form>
```

Cas « champs remplis » – données envoyées : nom = ben salah, prenom = AYA, salle = Salon, inscrire = ok

```
Bienvenue Aya BEN SALAH, vous travaillerez en salle Salon.
```

Cas « prénom vide » – données envoyées : nom = Ben Salah, prenom = , salle = Salon, inscrire = ok

```
Nom et prénom obligatoires.
```

Exercice 3.3 – Menu et fiche d'un plat (liens GET et include)

Écrire trois fichiers : `donnees.php` (tableau `$menu` : 3 plats avec clés numériques 1, 2, 3 et, pour chacun, `nom`, `prix`, `categorie` : Salade tunisienne 6 DT Entrée, Couscous à l'agneau 22 DT Plat, Makroudh 5 DT Dessert) ; `menu.php` qui inclut `donnees.php` et affiche chaque plat (boucle `for`) sous forme de **lien** `plat.php?num=...` ; `plat.php` qui affiche la fiche du plat demandé (nom, catégorie, prix) ou « Plat introuvable » (avec `die`) si le numéro est absent ou inconnu.

Solution :

Fichier : donnees.php

```php
<?php
$menu = [
    1 => ["nom" => "Salade tunisienne", "prix" => 6, "categorie" => "Entrée"],
    2 => ["nom" => "Couscous à l'agneau", "prix" => 22, "categorie" => "Plat"],
    3 => ["nom" => "Makroudh", "prix" => 5, "categorie" => "Dessert"],
];
?>
```

Fichier : menu.php

```php
<?php
require "donnees.php";
for ($num = 1; $num <= count($menu); $num++) {
    echo "<a href='plat.php?num=$num'>" . $menu[$num]["nom"] . "</a><br>";
}
?>
```

Cas « menu.php »

```
<a href='plat.php?num=1'>Salade tunisienne</a><br><a href='plat.php?num=2'>Couscous à
l'agneau</a><br><a href='plat.php?num=3'>Makroudh</a><br>
```

Fichier : plat.php

```php
<?php
require "donnees.php";
if (!isset($_GET["num"]) || !isset($menu[$_GET["num"]])) {
    die("Plat introuvable");
}
$p = $menu[$_GET["num"]];
echo "<h3>" . $p["nom"] . "</h3>";
echo "<p>Catégorie : " . $p["categorie"] . "<br>Prix : " . $p["prix"] . " DT</p>";
?>
```

Cas « clic sur le 2e lien » – données envoyées : num = 2

```
Couscous à l'agneau
Catégorie : Plat
Prix : 22 DT
```

Cas « plat.php?num=7 » – données envoyées : num = 7

```
Plat introuvable
```

Exercice 3.4 – Table de multiplication (GET)

Écrire `table.php` avec un formulaire **GET** (champ `n`, bouton `go`). Après envoi, si `n` n'est pas un nombre, afficher « Saisir un nombre. » ; sinon convertir `n` en entier et afficher la table de multiplication de `n` de 1 à 10 (`7 x 3 = 21`). Observer l'adresse de la page après l'envoi.

Solution :

Fichier : table.php

```
<?php
echo "<div id='resultat'>";
if (isset($_GET["go"])) {
    $n = $_GET["n"];
    if (!is_numeric($n)) {
        echo "<p>Saisir un nombre.</p>";
    } else {
        $n = (int)$n;
        echo "<p>Table de $n :</p>";
        for ($i = 1; $i <= 10; $i++) {
            echo $n . " x " . $i . " = " . $n * $i . "<br>";
        }
    }
}
echo "</div>";
?>
<form method="get">
  Nombre : <input type="text" name="n">
  <input type="submit" name="go" value="Afficher">
</form>
```

Cas « n = 7 » – données envoyées : n = 7, go = Afficher

```
Table de 7 :
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
```

Cas « n = abc » – données envoyées : n = abc, go = Afficher

```
Saisir un nombre.
```

→ Adresse obtenue : `table.php?n=7&go=Afficher` : les données sont visibles, contrairement à POST.

Partie 4 – Conversion, conditions et boucles

Exercice 4.1 – Prédire les types et les conversions

Écrire ce qu'affiche chaque `var_dump` (type **et** valeur).

```php
<?php
var_dump("7" + "3");
var_dump("7" . "3");
var_dump((int)"15 DT");
var_dump((int)4.9);
var_dump((bool)"0");
var_dump((bool)"false");
var_dump("10" == 10);
var_dump("10" === 10);
var_dump(is_numeric("1e3"));
?>
```

Solution :

Affichage :

```
int(10)
string(2) "73"
int(15)
int(4)
bool(false)
bool(true)
bool(true)
bool(false)
bool(true)
```

→ `+` convertit en nombres (10) ; `.` convertit en chaîne ("73"). `(int)` coupe à la première lettre et **tronque** 4.9 → 4.

→ `"0"` est faux mais `"false"` est une chaîne non vide donc **vraie**. == convertit, === compare aussi le type. `"1e3"` est numérique (notation scientifique).

Exercice 4.2 – Indice de masse corporelle (POST)

Écrire `imc.php` : formulaire POST (`poids` en kg, `taille` en m, bouton `calculer`). Vérifier avec `is_numeric` que les deux valeurs sont des nombres strictement positifs (sinon « Saisie incorrecte. »), convertir en réel, calculer `IMC = poids / (taille × taille)` arrondi à 1 décimale et afficher l'interprétation : moins de 18.5 « maigreur », moins de 25 « poids normal », moins de 30 « surpoids », sinon « obésité ».

Solution :

Fichier : imc.php

```
<?php
echo "<div id='resultat'>";
if (isset($_POST["calculer"])) {
    $poids = $_POST["poids"];
    $taille = $_POST["taille"];
    if (!is_numeric($poids) || !is_numeric($taille) || $poids <= 0 || $taille <= 0) {
        echo "<p>Saisie incorrecte.</p>";
    } else {
        $imc = round((float)$poids / ((float)$taille * (float)$taille), 1);
        if ($imc < 18.5) { $diag = "maigreur"; }
        elseif ($imc < 25) { $diag = "poids normal"; }
        elseif ($imc < 30) { $diag = "surpoids"; }
        else { $diag = "obésité"; }
        echo "<p>IMC = $imc : $diag</p>";
    }
}
echo "</div>";
?>
<form method="post">
  Poids (kg) : <input type="text" name="poids"><br>
  Taille (m) : <input type="text" name="taille"><br>
  <input type="submit" name="calculer" value="Calculer">
</form>
```

Cas « poids 70, taille 1.75 » – données envoyées : poids = 70, taille = 1.75, calculer = ok

```
IMC = 22.9 : poids normal
```

Cas « poids -5 » – données envoyées : poids = -5, taille = 1.75, calculer = ok

```
Saisie incorrecte.
```

Exercice 4.3 – Boucles

Écrire `boucles.php` qui affiche, avec les boucles demandées : (a) avec `for`, un triangle de 5 lignes d'étoiles (1 étoile sur la 1re ligne, 2 sur la 2e…, utiliser `str_repeat`) ; (b) avec `while`, la somme des entiers de 1 à 100 ; (c) avec `for`, les carrés des nombres de 1 à 10 sur une seule ligne, séparés par `-` (sans `-` après le dernier).

Solution :

Fichier : boucles.php

```
<?php
for ($i = 1; $i <= 5; $i++) {
    echo str_repeat("*", $i) . "<br>";
}
$somme = 0;
$k = 1;
while ($k <= 100) {
    $somme += $k;
    $k++;
}
echo "Somme : $somme<br>";
for ($i = 1; $i <= 10; $i++) {
    echo $i * $i;
    if ($i < 10) {
        echo " - ";
    }
}
?>
```

Affichage obtenu

```
*
**
***
****
*****
Somme : 5050
1 - 4 - 9 - 16 - 25 - 36 - 49 - 64 - 81 - 100
```

Exercice 4.4 – Listes déroulantes générées par des boucles

Écrire `date_naissance.php` qui affiche trois listes déroulantes générées **uniquement avec des boucles** : `jour` (1 à 31), `mois` (les noms des mois en français, stockés dans un tableau ; la `value` est le numéro du mois), `annee` (de l'année actuelle jusqu'à 1990, en décroissant ; l'année actuelle est présélectionnée avec `selected`).

Solution :

Fichier : date_naissance.php

```php
<?php
$noms = ["janvier", "février", "mars", "avril", "mai", "juin",
         "juillet", "août", "septembre", "octobre", "novembre", "décembre"];
echo "<select name='jour'>";
for ($j = 1; $j <= 31; $j++) {
    echo "<option value='$j'>$j</option>";
}
echo "</select>";
echo "<select name='mois'>";
for ($m = 1; $m <= 12; $m++) {
    echo "<option value='$m'>" . $noms[$m - 1] . "</option>";
}
echo "</select>";
echo "<select name='annee'>";
for ($a = date("Y"); $a >= 1990; $a--) {
    $sel = ($a == date("Y")) ? " selected" : "";
    echo "<option value='$a'$sel>$a</option>";
}
echo "</select>";
?>
```

Affichage obtenu

```
<select name='jour'>
<option value='1'>1</option>
<option value='2'>2</option>
<option value='3'>3</option>
<option value='4'>4</option>
<option value='5'>5</option>
<option value='6'>6</option>
<option value='7'>7</option>
<option value='8'>8</option>
<option value='9'>9</option>
[...]
```

Partie 5 – Tableaux et tableaux associatifs

Exercice 5.1 – Prédire l'affichage

Écrire ce que le navigateur affiche.

```
<?php
$s = ["nom" => "Rim", "age" => 17, "salle" => "Salon"];
$s["age"] = $s["age"] + 1;
$s["tel"] = "71 000 000";
unset($s["salle"]);
$cles = array_keys($s);
for ($i = 0; $i < count($cles); $i++) {
    echo $cles[$i] . "=" . $s[$cles[$i]] . ";";
}
echo count($s);
echo isset($s["salle"]) ? " oui" : " non";
$notes = [12, 8, 15];
$notes[] = 10;
sort($notes);
echo "<br>Notes :";
for ($i = 0; $i < count($notes); $i++) {
    echo " " . $notes[$i];
}
echo " | max " . max($notes) . " | 2e : " . $notes[1];
?>
```

Solution :

Affichage :

```
nom=Rim;age=18;tel=71 000 000;3 non
Notes : 8 10 12 15 | max 15 | 2e : 10
```

→ Après `unset`, la clé `salle` n'existe plus (`count` vaut 3, `isset` est faux). `array_keys` donne les clés `nom`, `age`, `tel` : la boucle `for` les parcourt dans l'ordre. `sort` réordonne les valeurs : `$notes[1]` est la 2e plus petite.

Exercice 5.2 – Statistiques sur un tableau indexé

Écrire `notes.php`. Avec `$notes = [12, 8, 15, 9.5, 17, 10];`, afficher : le nombre de notes, la moyenne (2 décimales), la meilleure et la plus faible note, le nombre de notes supérieures ou égales à 10 (boucle `for`), la liste triée par ordre **décroissant** (`rsort`, puis boucle `for` avec des virgules entre les notes, sans virgule après la dernière), puis « Admis » si la moyenne est supérieure ou égale à 10, « Ajourné » sinon.

Solution :

Fichier : notes.php

```
<?php
$notes = [12, 8, 15, 9.5, 17, 10];
$n = count($notes);
$moy = array_sum($notes) / $n;
$nb = 0;
for ($i = 0; $i < $n; $i++) {
    if ($notes[$i] >= 10) {
        $nb++;
    }
}
rsort($notes);
echo "Nombre : $n<br>";
echo "Moyenne : " . number_format($moy, 2) . "<br>";
echo "Max : " . max($notes) . " - Min : " . min($notes) . "<br>";
echo "Notes >= 10 : $nb<br>";
echo "Tri décroissant : ";
for ($i = 0; $i < $n; $i++) {
    echo $notes[$i];
    if ($i < $n - 1) {
        echo ", ";
    }
}
echo "<br>";
echo ($moy >= 10) ? "Admis" : "Ajourné";
?>
```

Affichage obtenu

```
Nombre : 6
Moyenne : 11.92
Max : 17 - Min : 8
Notes >= 10 : 4
Tri décroissant : 17, 15, 12, 10, 9.5, 8
Admis
```

Exercice 5.3 – Tableau de tableaux : bulletin d'une classe

Écrire `classe.php` avec le tableau `$eleves` suivant : Aya 14.5, Sami 9, Rim 16, Karim 11.5 (chaque élève est un tableau associatif `nom` / `moyenne`). Avec une boucle `for`, afficher un tableau HTML (nom, moyenne, mention : « Bien » à partir de 14, « Passable » à partir de 10, sinon « Insuffisant »), puis la moyenne de la classe, le nom du meilleur élève et le nombre d'élèves admis (moyenne ≥ 10).

Solution :

Fichier : classe.php

```
<?php
$eleves = [
    ["nom" => "Aya", "moyenne" => 14.5],
    ["nom" => "Sami", "moyenne" => 9],
    ["nom" => "Rim", "moyenne" => 16],
    ["nom" => "Karim", "moyenne" => 11.5],
];
$n = count($eleves);
$somme = 0;
$admis = 0;
$imax = 0;
echo "<table border='1'>";
for ($i = 0; $i < $n; $i++) {
    $moy = $eleves[$i]["moyenne"];
    if ($moy >= 14) { $mention = "Bien"; }
    elseif ($moy >= 10) { $mention = "Passable"; }
    else { $mention = "Insuffisant"; }
    echo "<tr><td>" . $eleves[$i]["nom"] . "</td><td>$moy</td><td>$mention</td></tr>";
    $somme += $moy;
    if ($moy >= 10) { $admis++; }
    if ($moy > $eleves[$imax]["moyenne"]) { $imax = $i; }
}
echo "</table>";
echo "Moyenne de la classe : " . $somme / $n . "<br>";
echo "Meilleur élève : " . $eleves[$imax]["nom"] . "<br>";
echo "Admis : $admis";
?>
```

Affichage obtenu

```
Aya | 14.5 | Bien
Sami | 9 | Insuffisant
Rim | 16 | Bien
Karim | 11.5 | Passable
Moyenne de la classe : 12.75
Meilleur élève : Rim
Admis : 3
```

→ On mémorise l'**indice** du meilleur élève (`$imax`) plutôt que de recopier le tableau.

Exercice 5.4 – Compter les occurrences (tableau associatif)

Avec le tableau `$villes = ["tunis", "sfax", "tunis", "sousse", "sfax", "tunis"];`, construire un tableau associatif `$compte` (clé = ville, valeur = nombre d'occurrences) avec une boucle `for` et `isset`. Afficher le résultat trié par nom de ville (`ksort`, puis `array_keys` et une boucle `for`) avec une majuscule initiale : « Sfax : 2 », « Sousse : 1 », « Tunis : 3 ».

Solution :

Fichier : villes.php

```
<?php
$villes = ["tunis", "sfax", "tunis", "sousse", "sfax", "tunis"];
$compte = [];
for ($i = 0; $i < count($villes); $i++) {
    $v = $villes[$i];
    if (isset($compte[$v])) {
        $compte[$v]++;
    } else {
        $compte[$v] = 1;
    }
}
ksort($compte);
$cles = array_keys($compte);
for ($i = 0; $i < count($cles); $i++) {
    echo ucfirst($cles[$i]) . " : " . $compte[$cles[$i]] . "<br>";
}
?>
```

Affichage obtenu

```
Sfax : 2
Sousse : 1
Tunis : 3
```

Partie 6 – Les dates

Exercice 6.1 – Formats de date

On pose `$t = mktime(9, 5, 30, 12, 25, 2026);` (vendredi 25 décembre 2026, 9 h 05 min 30 s). **(1)** Écrire le format `date()` qui affiche : (a) `2026-12-25` ; (b) `25-12-26` ; (c) `9h05` ; (d) `Fri, 25 Dec` ; (e) `09:05:30`. **(2)** Prédire l'affichage du script ci-dessous.

```
<?php
$t = mktime(9, 5, 30, 12, 25, 2026);
echo date("d/m/Y", $t) . "<br>";
echo date("D j M y", $t) . "<br>";
echo date("H:m:s", $t) . "<br>";
echo date("g:i a", $t) . "<br>";
echo date("N w z", $t) . "<br>";
echo date("t L", $t);
?>
```

Solution :

(1) Formats : `Y-m-d` ; `d-m-y` ; `G\hi` (le h échappé par `\`, entre guillemets simples) ; `D, d M` ; `H:i:s`. Vérification :

```
<?php
$t = mktime(9, 5, 30, 12, 25, 2026);
echo date("Y-m-d", $t) . "<br>";
echo date("d-m-y", $t) . "<br>";
echo date('G\hi', $t) . "<br>";
echo date("D, d M", $t) . "<br>";
echo date("H:i:s", $t) . "<br>";
?>
```

```
2026-12-25
25-12-26
9h05
Fri, 25 Dec
09:05:30
```

(2) Affichage du script :

```php
<?php
$t = mktime(9, 5, 30, 12, 25, 2026);
echo date("d/m/Y", $t) . "<br>";
echo date("D j M y", $t) . "<br>";
echo date("H:m:s", $t) . "<br>";
echo date("g:i a", $t) . "<br>";
echo date("N w z", $t) . "<br>";
echo date("t L", $t);
?>
```

```
25/12/2026
Fri 25 Dec 26
09:12:30
9:05 am
5 5 358
31 0
```

`H:m:s` affiche le **mois** (12) à la place des minutes : les minutes s'écrivent `i`. `N` = 5 (vendredi, lundi = 1), `w` = 5 (dimanche = 0), `z` = 358 (jour de l'année, à partir de 0), `t` = 31 jours dans le mois, `L` = 0 (2026 n'est pas bissextile).

Exercice 6.2 – La date du jour en français

Écrire `aujourdhui.php` qui affiche la date du jour sous la forme « Aujourd'hui : samedi 14 novembre 2026 » (jour de la semaine et mois **en français**, à l'aide de deux tableaux et de `date("w")`, `date("j")`, `date("n")`, `date("Y")`).

Solution :

Fichier : aujourdhui.php

```php
<?php
date_default_timezone_set("Africa/Tunis");
$jours = ["dimanche", "lundi", "mardi", "mercredi", "jeudi", "vendredi", "samedi"];
$mois = ["", "janvier", "février", "mars", "avril", "mai", "juin",
         "juillet", "août", "septembre", "octobre", "novembre", "décembre"];
echo "Aujourd'hui : " . $jours[date("w")] . " " . date("j") . " ";
echo $mois[date("n")] . " " . date("Y");
?>
```

Cas « exécuté le 14/11/2026 »

```
Aujourd'hui : samedi 14 novembre 2026
```

→ Le tableau `$mois` commence par une case vide pour que `$mois[11]` soit novembre (`date("n")` va de 1 à 12).

Exercice 6.3 – Calcul de l'âge (formulaire avec bouton radio)

Écrire `age.php` (POST) : champ `naissance` (`type="date"`), deux boutons radio `titre` (Monsieur / Madame) et le bouton `calculer`. Si la date ou la civilité manque, afficher « Choisissez une civilité et une date de naissance. ». Sinon afficher « Monsieur, vous avez 18 ans (majeur). » (ou « (mineur) ») puis « Prochain anniversaire dans N jour(s). ». La date arrive au format `AAAA-MM-JJ` : extraire l'année, le mois et le jour avec `substr` . Attention : l'âge est la différence des années, **moins 1** si l'anniversaire de cette année n'est pas encore passé (comparer `date("md")` avec le mois et le jour de naissance collés).

Solution :

Fichier : age.php

```
<?php
date_default_timezone_set("Africa/Tunis");
echo "<div id='resultat'>";
if (isset($_POST["calculer"])) {
    if (empty($_POST["naissance"]) || !isset($_POST["titre"])) {
        echo "<p>Choisissez une civilité et une date de naissance.</p>";
    } else {
        $naissance = $_POST["naissance"];
        $an = substr($naissance, 0, 4);
        $mois = substr($naissance, 5, 2);
        $jour = substr($naissance, 8, 2);
        $age = date("Y") - $an;
        if (date("md") < $mois . $jour) {
            $age--;
        }
        $prochain = date("Y") . "-" . $mois . "-" . $jour;
        if ($prochain < date("Y-m-d")) {
            $prochain = (date("Y") + 1) . "-" . $mois . "-" . $jour;
        }
        $reste = round((strtotime($prochain) - strtotime(date("Y-m-d"))) / 86400);
        echo "<p>" . $_POST["titre"] . ", vous avez $age ans ";
        echo ($age >= 18) ? "(majeur)" : "(mineur)";
        echo ".</p><p>Prochain anniversaire dans $reste jour(s).</p>";
    }
}
echo "</div>";
?>
<form method="post">
  <input type="radio" name="titre" value="Monsieur"> Monsieur
  <input type="radio" name="titre" value="Madame"> Madame<br>
  Naissance : <input type="date" name="naissance"><br>
  <input type="submit" name="calculer" value="Calculer">
</form>
```

Cas « au 14/11/2026 : Monsieur, né le 2008-05-12 » – données envoyées : titre = Monsieur, naissance = 2008-05-12, calculer = ok

```
Monsieur, vous avez 18 ans (majeur).
Prochain anniversaire dans 179 jour(s).
```

Cas « au 14/11/2026 : Madame, née le 2010-12-25 » – données envoyées : titre = Madame, naissance = 2010-12-25, calculer = ok

```
Madame, vous avez 15 ans (mineur).
Prochain anniversaire dans 41 jour(s).
```

Cas « civilité non cochée » – données envoyées : naissance = 2010-12-25, calculer = ok

```
Choisissez une civilité et une date de naissance.
```

→ `date("md")` donne 1114 le 14 novembre ; pour une naissance un 12 mai, on compare à `"0512"` : comme 1114 est plus grand, l'anniversaire est passé et on ne retire rien. Pour un 25 décembre (`"1225"`), `1114 < 1225` : l'anniversaire n'est pas encore passé, on retire 1. Dates et jours calculés au 14/11/2026.

Exercice 6.4 – Différence entre deux dates

Écrire `difference.php` (POST) : deux champs `d1` et `d2` de type date et le bouton `calculer`. Si une date manque : « Deux dates sont nécessaires. » ; si `d2` est avant `d1` : « La 2e date doit être postérieure à la 1re. » ; sinon afficher le nombre de **jours** entre les deux dates, puis le même écart en « semaines et jours » (par exemple `42 semaine(s) et 1 jour(s)`).

Solution :

Fichier : difference.php

```
<?php
echo "<div id='resultat'>";
if (isset($_POST["calculer"])) {
    if (empty($_POST["d1"]) || empty($_POST["d2"])) {
        echo "<p>Deux dates sont nécessaires.</p>";
    } else {
        $t1 = strtotime($_POST["d1"]);
        $t2 = strtotime($_POST["d2"]);
        if ($t2 < $t1) {
            echo "<p>La 2e date doit être postérieure à la 1re.</p>";
        } else {
            $jours = round(($t2 - $t1) / 86400);
            echo "<p>$jours jour(s) d'écart.</p>";
            echo "<p>Soit " . floor($jours / 7) . " semaine(s) et ";
            echo $jours % 7 . " jour(s).</p>";
        }
    }
}
echo "</div>";
?>
<form method="post">
  Du : <input type="date" name="d1"> au : <input type="date" name="d2">
  <input type="submit" name="calculer" value="Calculer">
</form>
```

Cas « 2026-03-05 et 2026-12-25 » – données envoyées : d1 = 2026-03-05, d2 = 2026-12-25, calculer = ok

```
295 jour(s) d'écart.
Soit 42 semaine(s) et 1 jour(s).
```

Cas « dates inversées » – données envoyées : d1 = 2026-12-25, d2 = 2026-03-05, calculer = ok

```
La 2e date doit être postérieure à la 1re.
```

Exercice 6.5 – Date de livraison

Écrire `livraison.php` (POST) : champs `date` (date de commande, type date), `delai` (nombre de jours, type number) et bouton `calculer`. La date de livraison est la date de commande **plus** `delai` jours ; si elle tombe un **dimanche**, la livraison est reportée au lundi. Afficher « Livraison le lundi 16/03/2026. » (jour de la semaine en français, date au format `jj/mm/aaaa`).

Solution :

Fichier : livraison.php

```
<?php
date_default_timezone_set("Africa/Tunis");
$jours = ["dimanche", "lundi", "mardi", "mercredi", "jeudi", "vendredi", "samedi"];
echo "<div id='resultat'>";
if (isset($_POST["calculer"])) {
    if (empty($_POST["date"]) || !is_numeric($_POST["delai"])) {
        echo "<p>Saisie incorrecte.</p>";
    } else {
        $delai = (int)$_POST["delai"];
        $t = strtotime($_POST["date"] . " +" . $delai . " days");
        if (date("w", $t) == 0) {
            $t = strtotime("+1 day", $t);
        }
        echo "<p>Livraison le " . $jours[date("w", $t)] . " " . date("d/m/Y", $t) . ".</p>";
    }
}
echo "</div>";
?>
<form method="post">
  Commande : <input type="date" name="date">
  Délai : <input type="number" name="delai">
  <input type="submit" name="calculer" value="Calculer">
</form>
```

Cas « 2026-03-05 + 10 jours (le 15/03 est un dimanche) » – données envoyées : date = 2026-03-05, delai = 10, calculer = ok

```
Livraison le lundi 16/03/2026.
```

Cas « 2026-03-05 + 3 jours » – données envoyées : date = 2026-03-05, delai = 3, calculer = ok

```
Livraison le lundi 09/03/2026.
```

Exercice 6.6 – Calendrier d'un mois

Écrire `mois.php` (POST) : liste déroulante `mois` (noms français, `value` de 1 à 12), champ `annee` et bouton `voir`. Si l'année n'est pas un nombre, afficher « Année invalide. » ; sinon afficher : le nombre de jours du mois, le jour de la semaine du 1er du mois (en français) et si l'année est bissextile. Exemple : « Mars 2026 : 31 jours. Le 1er est un dimanche. Année bissextile : non. »

Solution :

Fichier : mois.php

```
<?php
$jours = ["dimanche", "lundi", "mardi", "mercredi", "jeudi", "vendredi", "samedi"];
$noms = ["", "Janvier", "Février", "Mars", "Avril", "Mai", "Juin",
         "Juillet", "Août", "Septembre", "Octobre", "Novembre", "Décembre"];
echo "<div id='resultat'>";
if (isset($_POST["voir"])) {
    $mois = (int)$_POST["mois"];
    $annee = $_POST["annee"];
    if (!is_numeric($annee)) {
        echo "<p>Année invalide.</p>";
    } else {
        $t = mktime(0, 0, 0, $mois, 1, (int)$annee);
        echo "<p>" . $noms[$mois] . " $annee : " . date("t", $t) . " jours.</p>";
        echo "<p>Le 1er est un " . $jours[date("w", $t)] . ".</p>";
        echo "<p>Année bissextile : " . (date("L", $t) ? "oui" : "non") . "</p>";
    }
}
echo "</div>";
?>
<form method="post">
  <select name="mois">
<?php
for ($m = 1; $m <= 12; $m++) {
    echo "    <option value='$m'>" . $noms[$m] . "</option>\n";
}
?>
  </select>
  Année : <input type="text" name="annee">
  <input type="submit" name="voir" value="Voir">
</form>
```

Cas « mois = 3, annee = 2026 » – données envoyées : mois = 3, annee = 2026, voir = ok

```
Mars 2026 : 31 jours.
Le 1er est un dimanche.
Année bissextile : non
```

Cas « mois = 2, annee = 2028 » – données envoyées : mois = 2, annee = 2028, voir = ok

```
Février 2028 : 29 jours.
Le 1er est un mardi.
Année bissextile : oui
```

Exercice 6.7 – Changer le format d'une date

Sans formulaire . (a) Convertir `$fr = "25/12/2026";` en `2026-12-25` de **deux** façons : en extrayant le jour, le mois et l'année avec `substr` puis en les recollant ; avec `mktime` puis `date`. (b) Convertir `$sql = "2026-03-05";` en `05/03/2026` avec `date` et `strtotime`. (c) Pourquoi ne peut-on pas écrire `strtotime("25/12/2026")` ? (d) Vérifier avec `checkdate` si `"31/02/2026"` existe (attention à l'ordre des arguments).

Solution :

Fichier : formats_date.php

```
<?php
$fr = "25/12/2026";
$j = substr($fr, 0, 2);
$m = substr($fr, 3, 2);
$a = substr($fr, 6, 4);
echo "Avec substr : " . $a . "-" . $m . "-" . $j . "<br>";
echo "Avec mktime : " . date("Y-m-d", mktime(0, 0, 0, $m, $j, $a)) . "<br>";
$sql = "2026-03-05";
echo "Date française : " . date("d/m/Y", strtotime($sql)) . "<br>";
var_dump(strtotime($fr));
$test = "31/02/2026";
$j = substr($test, 0, 2);
$m = substr($test, 3, 2);
$a = substr($test, 6, 4);
echo checkdate($m, $j, $a) ? "$test est valide" : "$test n'existe pas";
?>
```

Affichage obtenu

```
Avec substr : 2026-12-25
Avec mktime : 2026-12-25
Date française : 05/03/2026
bool(false)
31/02/2026 n'existe pas
```

→ `substr($fr, début, longueur)` : les positions commencent à 0 (`25/12/2026` : jour en 0, mois en 3, année en 6).

→ (c) `strtotime` lit `mm/jj/aaaa` : `25/12/2026` serait « mois 25 » → échec, `false`.

→ (d) `checkdate(MOIS, JOUR, ANNÉE)` : le mois **d'abord**. `checkdate($j, $m, $a)` donnerait un mauvais résultat.

Partie 7 – Formulaires avec boutons radio

Exercice 7.1 – Civilité (boutons radio)

Écrire `bonjour.php` (POST) : champs `prenom` et `nom` (texte), boutons radio `civilite` (Monsieur / Madame) et bouton `envoyer`. Après envoi, si aucune civilité n'est cochée ou si un champ est vide : « Veuillez tout remplir. » ; sinon « Bonjour Madame Prénom NOM ». Quelle valeur contient `$_POST["civilite"]` ? Que se passe-t-il si aucun bouton n'est coché ?

Solution :

Fichier : bonjour.php

```
<?php
echo "<div id='resultat'>";
if (isset($_POST["envoyer"])) {
    if (!isset($_POST["civilite"]) || empty($_POST["prenom"]) || empty($_POST["nom"])) {
        echo "<p>Veuillez tout remplir.</p>";
    } else {
        echo "<p>Bonjour " . $_POST["civilite"] . " " . ucfirst(strtolower($_POST["prenom"]));
        echo " " . strtoupper($_POST["nom"]) . "</p>";
    }
}
echo "</div>";
?>
<form method="post">
  <input type="radio" name="civilite" value="Monsieur"> Monsieur
  <input type="radio" name="civilite" value="Madame"> Madame<br>
  Prénom : <input type="text" name="prenom"><br>
  Nom : <input type="text" name="nom"><br>
  <input type="submit" name="envoyer" value="Envoyer">
</form>
```

Cas « civilité cochée » – données envoyées : civilite = Madame, prenom = aya, nom = Ben Salah, envoyer = ok

```
Bonjour Madame Aya BEN SALAH
```

Cas « aucune civilité cochée » – données envoyées : prenom = aya, nom = Ben Salah, envoyer = ok

```
Veuillez tout remplir.
```

→ `$_POST["civilite"]` contient la `value` du bouton coché (`Monsieur` ou `Madame`), pas le texte affiché à côté. Si aucun bouton n'est coché, la clé n'existe pas : lire `$_POST["civilite"]` sans `isset` donnerait « Undefined array key ».

Exercice 7.2 – Calculatrice à boutons radio

Écrire `calcul.php` (POST) : champs `a` et `b`, cinq boutons radio `op` (`+`, `-`, `*`, `/`, `%` pour le reste), bouton `calculer`. Vérifier les saisies avec `is_numeric` et la présence d'une opération (sinon « Saisie incorrecte. »). Utiliser `switch`. Pour `/` et `%`, afficher « Division par zéro impossible. » si `b` vaut 0. Les valeurs saisies et l'opération choisie doivent **rester affichées** après l'envoi (`checked` conditionnel). Exemple : « 17 % 5 = 2 ».

Solution :

Fichier : calcul.php

```
<?php
$a = "";
$b = "";
$op = "";
echo "<div id='resultat'>";
if (isset($_POST["calculer"])) {
    $a = $_POST["a"];
    $b = $_POST["b"];
    if (isset($_POST["op"]) && is_numeric($a) && is_numeric($b)) {
        $op = $_POST["op"];
        if (($op == "/" || $op == "%") && $b == 0) {
            echo "<p>Division par zéro impossible.</p>";
        } else {
            switch ($op) {
                case "+": $r = $a + $b; break;
                case "-": $r = $a - $b; break;
                case "*": $r = $a * $b; break;
                case "/": $r = $a / $b; break;
                case "%": $r = $a % $b; break;
            }
            echo "<p>$a $op $b = $r</p>";
        }
    } else {
        echo "<p>Saisie incorrecte.</p>";
    }
}
echo "</div>";
?>
<form method="post">
  <input type="text" name="a" value="<?= htmlspecialchars($a) ?>">
<?php
$ops = ["+", "-", "*", "/", "%"];
for ($i = 0; $i < count($ops); $i++) {
    $c = ($op == $ops[$i]) ? " checked" : "";
    echo "  <input type='radio' name='op' value='" . $ops[$i] . "'$c> " . $ops[$i] . "\n";
}
?>
  <input type="text" name="b" value="<?= htmlspecialchars($b) ?>">
  <input type="submit" name="calculer" value="=">
</form>
```

Cas « 17 % 5 » – données envoyées : a = 17, b = 5, op = %, calculer = =

```
17 % 5 = 2
```

Cas « 9 / 0 » – données envoyées : a = 9, b = 0, op = /, calculer = =

```
Division par zéro impossible.
```

Cas « aucune opération » – données envoyées : a = 9, b = 2, calculer = =

```
Saisie incorrecte.
```

Exercice 7.3 – QCM de 5 questions (boutons radio)

Écrire `qcm.php` (POST) : le formulaire est **généré** par une boucle à partir d'un tableau `$questions` (chaque question : énoncé, 3 choix, indice de la bonne réponse). Les boutons radio de la question *i* s'appellent `q0`, `q1`… et ont pour `value` l'indice du choix. Après `corriger`, afficher « Score : N / 5 » et une mention : 5 « Excellent », 3 ou 4 « Bien », sinon « À revoir ». Une question sans réponse compte 0. Proposer vos 5 questions sur GET/POST, define, include, die, mysqli.

Solution :

Fichier : qcm.php

```php
<?php
$questions = [
    ["Données visibles dans l'adresse :", ["GET", "POST", "FILES"], 0],
    ["Arrête le script :", ["echo", "die", "include"], 1],
    ["Type d'une valeur reçue d'un formulaire :", ["int", "bool", "string"], 2],
    ["Opérateur de concaténation :", ["+", ".", "&"], 1],
    ["Nombre de lignes d'un SELECT :", ["num_rows", "affected_rows", "fetch_row"], 0],
];
if (isset($_POST["corriger"])) {
    $score = 0;
    for ($i = 0; $i < count($questions); $i++) {
        if (isset($_POST["q$i"]) && $_POST["q$i"] == $questions[$i][2]) {
            $score++;
        }
    }
    echo "<div id='resultat'><p>Score : $score / " . count($questions) . "</p>";
    if ($score == 5) { echo "<p>Excellent</p>"; }
    elseif ($score >= 3) { echo "<p>Bien</p>"; }
    else { echo "<p>À revoir</p>"; }
    echo "</div>";
}
echo "<form method='post'>";
for ($i = 0; $i < count($questions); $i++) {
    echo "<p>" . ($i + 1) . ") " . $questions[$i][0] . "<br>";
    for ($j = 0; $j < 3; $j++) {
        echo "<input type='radio' name='q$i' value='$j'> " . $questions[$i][1][$j] . " ";
    }
    echo "</p>";
}
echo "<input type='submit' name='corriger' value='Corriger'></form>";
?>
```

Cas « 4 bonnes réponses, la 5e sans réponse » – données envoyées : q0 = 0, q1 = 1, q2 = 2, q3 = 1, corriger = ok

```
Score : 4 / 5
Bien
```

Cas « une seule bonne réponse » – données envoyées : q0 = 0, q1 = 0, q2 = 0, q3 = 0, q4 = 1, corriger = ok

```
Score : 1 / 5
À revoir
```

→ Les `value` sont des indices : `"1" == 1` est vrai (comparaison avec conversion). `isset` évite l'erreur quand une question reste sans réponse.

Exercice 7.4 – Convertisseur de monnaie (radio et constantes)

Écrire `convertisseur.php` (POST) : champ `montant`, trois boutons radio `sens` (`dt_eur` : DT → EUR, `eur_dt` : EUR → DT, `dt_usd` : DT → USD), bouton `convertir`. Définir avec `define` les constantes `DT_EUR` (0.30) et `DT_USD` (0.32) (taux **fictifs**). Afficher le résultat avec 2 décimales, par exemple « 100 → 30.00 EUR ». Le montant et le choix doivent rester affichés après l'envoi.

Solution :

Fichier : convertisseur.php

```
<?php
define("DT_EUR", 0.30);
define("DT_USD", 0.32);
$montant = "";
$sens = "";
echo "<div id='resultat'>";
if (isset($_POST["convertir"])) {
    $montant = $_POST["montant"];
    if (!is_numeric($montant) || !isset($_POST["sens"])) {
        echo "<p>Saisie incorrecte.</p>";
    } else {
        $sens = $_POST["sens"];
        switch ($sens) {
            case "dt_eur": $r = $montant * DT_EUR; $unite = "EUR"; break;
            case "eur_dt": $r = $montant / DT_EUR; $unite = "DT"; break;
            case "dt_usd": $r = $montant * DT_USD; $unite = "USD"; break;
        }
        echo "<p>$montant → " . number_format($r, 2) . " $unite</p>";
    }
}
echo "</div>";
?>
<form method="post">
  Montant : <input type="text" name="montant" value="<?= htmlspecialchars($montant) ?>"><br>
<?php
$valeurs = ["dt_eur", "eur_dt", "dt_usd"];
$textes = ["DT → EUR", "EUR → DT", "DT → USD"];
for ($i = 0; $i < 3; $i++) {
    $c = ($sens == $valeurs[$i]) ? " checked" : "";
    echo "  <input type='radio' name='sens' value='" . $valeurs[$i] . "'$c>";
    echo " " . $textes[$i] . "\n";
}
?>
  <input type="submit" name="convertir" value="Convertir">
</form>
```

Cas « 100 DT → EUR » – données envoyées : montant = 100, sens = dt_eur, convertir = ok

```
100 → 30.00 EUR
```

Cas « 60 EUR → DT » – données envoyées : montant = 60, sens = eur_dt, convertir = ok

```
60 → 200.00 DT
```

Exercice 7.5 – Commande au restaurant (radio, quantité et remise)

Écrire `commande.php` (POST) avec `$menu = [1 => ["Salade tunisienne", 6], 2 => ["Couscous à l'agneau", 22], 3 => ["Makroudh", 5]];`. Le formulaire contient : des boutons radio `plat` **générés par une boucle** `for` (value = clé du tableau), un champ `qte` (nombre, 1 par défaut), deux boutons radio `mode` (`place` coché par défaut, `emporter`) et le bouton `commander`. Après envoi : si un champ est incorrect (`qte` non numérique ou ≤ 0, plat non choisi) « Veuillez remplir tout le formulaire. » ; sinon afficher la ligne « 3 x Couscous à l'agneau = 66 DT », puis, pour `emporter`, « Remise à emporter (10 %) » et enfin « Total à payer : 59.40 DT ».

Solution :

Fichier : commande.php

```
<?php
$menu = [1 => ["Salade tunisienne", 6], 2 => ["Couscous à l'agneau", 22],
         3 => ["Makroudh", 5]];
echo "<div id='resultat'>";
if (isset($_POST["commander"])) {
    if (isset($_POST["plat"]) && isset($_POST["mode"])
        && is_numeric($_POST["qte"]) && $_POST["qte"] > 0) {
        $p = $menu[$_POST["plat"]];
        $qte = (int)$_POST["qte"];
        $total = $p[1] * $qte;
        echo "<p>$qte x " . $p[0] . " = $total DT</p>";
        if ($_POST["mode"] == "emporter") {
            $total = $total * 0.9;
            echo "<p>Remise à emporter (10 %)</p>";
        }
        echo "<p>Total à payer : " . number_format($total, 2) . " DT</p>";
    } else {
        echo "<p>Veuillez remplir tout le formulaire.</p>";
    }
}
echo "</div>";
?>
<form method="post">
<?php
for ($num = 1; $num <= count($menu); $num++) {
    echo "<input type='radio' name='plat' value='$num'> " . $menu[$num][0];
    echo " (" . $menu[$num][1] . " DT)<br>";
}
?>
  Quantité : <input type="number" name="qte" value="1"><br>
  <input type="radio" name="mode" value="place" checked> Sur place
  <input type="radio" name="mode" value="emporter"> À emporter
  <input type="submit" name="commander" value="Commander">
</form>
```

Cas « Couscous, 3, à emporter » – données envoyées : plat = 2, qte = 3, mode = emporter, commander = ok

```
3 x Couscous à l'agneau = 66 DT
Remise à emporter (10 %)
Total à payer : 59.40 DT
```

Cas « Makroudh, 2, sur place » – données envoyées : plat = 3, qte = 2, mode = place, commander = ok

```
2 x Makroudh = 10 DT
Total à payer : 10.00 DT
```

Cas « aucun plat choisi » – données envoyées : qte = 2, mode = place, commander = ok

```
Veuillez remplir tout le formulaire.
```

Partie 8 – MySQLi procédural (base Dar Zitouna)

Exercice 8.1 – Prédire les résultats (num_rows, fetch, affected_rows)

Avec les tables de l'annexe (`SERVEUR`, `COMMANDE`), écrire ce que le navigateur affiche. La base est réinitialisée avant le script.

```php
<?php
require "config.php";
$cnx = mysqli_connect(HOTE, UTIL, MDP, BASE) or die("Connexion impossible");
$res = mysqli_query($cnx, "SELECT nom, salle FROM SERVEUR WHERE salle = 'Salon'");
echo mysqli_num_rows($res) . "<br>";
$l = mysqli_fetch_row($res);
echo $l[0] . "<br>";
$l = mysqli_fetch_array($res);
echo $l["nom"] . " " . $l["salle"] . "<br>";
var_dump(mysqli_fetch_row($res));
$sql = "SELECT plat, prix FROM COMMANDE WHERE categorie = 'Dessert' ORDER BY prix";
$res = mysqli_query($cnx, $sql);
$l = mysqli_fetch_array($res);
echo $l["plat"] . " " . $l["prix"] . "<br>";
mysqli_query($cnx, "UPDATE COMMANDE SET prix = prix + 1 WHERE categorie = 'Dessert'");
echo mysqli_affected_rows($cnx) . "<br>";
mysqli_query($cnx, "UPDATE COMMANDE SET qte = 2 WHERE num = 11");
echo mysqli_affected_rows($cnx) . "<br>";
mysqli_query($cnx, "DELETE FROM COMMANDE WHERE categorie = 'Boisson'");
echo mysqli_affected_rows($cnx);
?>
```

Solution :

Affichage :

```
2
Rim
Nour Salon
NULL
Makroudh 5.00
2
0
3
```

→ Les deux serveurs du Salon : Rim puis Nour (un appel de `fetch` = une ligne). Quand il n'y a plus de ligne, `mysqli_fetch_row` renvoie `NULL` (valeur fausse : c'est ce qui arrête une boucle `while`).

→ `prix` est un `DECIMAL(5,2)` : PHP le reçoit sous la forme de la chaîne `5.00`.

→ L'`UPDATE` du prix modifie 2 lignes. Celui de `qte` ne change rien (la commande 11 a déjà `qte = 2`) : `affected_rows` vaut **0**. Le `DELETE` supprime les 3 boissons.

Exercice 8.2 – Liste des serveurs

Écrire `serveurs.php` : se connecter avec `require "config.php"` et `or die(...)`, puis afficher dans un tableau HTML le numéro, le nom, la salle et la date d'embauche au format `jj/mm/aaaa` de chaque serveur (`mysqli_fetch_row`), précédé de « N serveur(s) » (`mysqli_num_rows`).

Solution :

Fichier : serveurs.php

```
<?php
require "config.php";
$cnx = mysqli_connect(HOTE, UTIL, MDP, BASE) or die("Connexion impossible");
mysqli_set_charset($cnx, "utf8mb4");
$res = mysqli_query($cnx, "SELECT num, nom, salle, date_embauche FROM SERVEUR");
echo "<p>" . mysqli_num_rows($res) . " serveur(s)</p>";
echo "<table border='1'>";
while ($l = mysqli_fetch_row($res)) {
    echo "<tr><td>" . $l[0] . "</td><td>" . $l[1] . "</td><td>" . $l[2] . "</td>";
    echo "<td>" . date("d/m/Y", strtotime($l[3])) . "</td></tr>";
}
echo "</table>";
mysqli_close($cnx);
?>
```

Affichage obtenu

```
4 serveur(s)
1 | Sami | Terrasse | 01/01/2022
2 | Rim | Salon | 01/09/2023
3 | Karim | Terrasse | 15/02/2019
4 | Nour | Salon | 10/01/2024
```

Exercice 8.3 – Serveurs d'une salle (boutons radio générés par la base)

Écrire `salles.php` (POST). Les boutons radio `salle` sont **générés** par une requête `SELECT DISTINCT salle FROM SERVEUR` (avec `mysqli_fetch_row`). Après envoi (`afficher`), afficher le nombre de serveurs de la salle choisie, puis, pour chacun (avec `mysqli_fetch_array`), son nom et le nombre de commandes qu'il a prises. Utiliser la requête `SELECT nom, (SELECT COUNT(*) FROM COMMANDE WHERE COMMANDE.numserveur = SERVEUR.num) AS nb FROM SERVEUR WHERE salle = '$salle'`. Sans salle choisie : « Choisissez une salle. »

Solution :

Fichier : salles.php

```
<?php
require "config.php";
$cnx = mysqli_connect(HOTE, UTIL, MDP, BASE) or die("Connexion impossible");
mysqli_set_charset($cnx, "utf8mb4");
echo "<form method='post'>";
$res = mysqli_query($cnx, "SELECT DISTINCT salle FROM SERVEUR ORDER BY salle");
while ($l = mysqli_fetch_row($res)) {
    echo "<input type='radio' name='salle' value='" . $l[0] . "'> " . $l[0] . " ";
}
echo "<input type='submit' name='afficher' value='Afficher'></form>";
echo "<div id='resultat'>";
if (isset($_POST["afficher"])) {
    if (!isset($_POST["salle"])) {
        echo "<p>Choisissez une salle.</p>";
    } else {
        $salle = $_POST["salle"];
        $sql = "SELECT nom, (SELECT COUNT(*) FROM COMMANDE";
        $sql .= " WHERE COMMANDE.numserveur = SERVEUR.num) AS nb";
        $sql .= " FROM SERVEUR WHERE salle = '$salle'";
        $res = mysqli_query($cnx, $sql);
        echo "<p>" . mysqli_num_rows($res) . " serveur(s) en $salle</p>";
        while ($l = mysqli_fetch_array($res)) {
            echo $l["nom"] . " : " . $l["nb"] . " commande(s)<br>";
        }
    }
}
echo "</div>";
mysqli_close($cnx);
?>
```

Cas « salle = Terrasse » – données envoyées : salle = Terrasse, afficher = ok

```
2 serveur(s) en Terrasse
Sami : 4 commande(s)
Karim : 4 commande(s)
```

Cas « salle = Salon » – données envoyées : salle = Salon, afficher = ok

```
2 serveur(s) en Salon
Rim : 4 commande(s)
Nour : 0 commande(s)
```

Cas « aucune salle choisie » – données envoyées : afficher = ok

```
Choisissez une salle.
```

Exercice 8.4 – Ajouter un serveur (INSERT et affected_rows)

Écrire `ajout_serveur.php` (POST) : champs `num`, `nom`, boutons radio `salle` (Terrasse / Salon), champ `date` (type date) et bouton `ajouter`. Après envoi, si un champ manque ou si `num` n'est pas numérique : « Veuillez remplir tous les champs. » ; sinon exécuter un `INSERT` ; si `mysqli_query` renvoie `true` et que `mysqli_affected_rows` vaut 1 : « Serveur ajouté. » ; sinon « Échec : ce numéro existe déjà. ». Terminer en affichant le nombre total de serveurs (`mysqli_num_rows`). On suppose que le nom ne contient pas d'apostrophe.

Solution :

Fichier : ajout_serveur.php

```
<?php
require "config.php";
$cnx = mysqli_connect(HOTE, UTIL, MDP, BASE) or die("Connexion impossible");
mysqli_set_charset($cnx, "utf8mb4");
echo "<div id='resultat'>";
if (isset($_POST["ajouter"])) {
    if (isset($_POST["salle"]) && is_numeric($_POST["num"])
        && !empty($_POST["nom"]) && !empty($_POST["date"])) {
        $num = (int)$_POST["num"];
        $nom = $_POST["nom"];
        $salle = $_POST["salle"];
        $date = $_POST["date"];
        $sql = "INSERT INTO SERVEUR VALUES ($num, '$nom', '$salle', '$date')";
        $ok = mysqli_query($cnx, $sql);
        if ($ok && mysqli_affected_rows($cnx) == 1) {
            echo "<p>Serveur ajouté.</p>";
        } else {
            echo "<p>Échec : ce numéro existe déjà.</p>";
        }
    } else {
        echo "<p>Veuillez remplir tous les champs.</p>";
    }
}
$res = mysqli_query($cnx, "SELECT * FROM SERVEUR");
echo "<p>Total : " . mysqli_num_rows($res) . " serveur(s)</p>";
echo "</div>";
mysqli_close($cnx);
?>
<form method="post">
  N° : <input type="text" name="num"> Nom : <input type="text" name="nom"><br>
  <input type="radio" name="salle" value="Terrasse"> Terrasse
  <input type="radio" name="salle" value="Salon"> Salon<br>
  Embauché le : <input type="date" name="date">
  <input type="submit" name="ajouter" value="Ajouter">
</form>
```

Cas « num 5, Aya, Salon, 2025-09-01 » – données envoyées : num = 5, nom = Aya, salle = Salon, date = 2025-09-01, ajouter = ok

```
Serveur ajouté.
Total : 5 serveur(s)
```

Cas « num 1 (déjà utilisé) » – données envoyées : num = 1, nom = Zied, salle = Salon, date = 2025-09-01, ajouter = ok

```
Échec : ce numéro existe déjà.
Total : 4 serveur(s)
```

Cas « radio non coché » – données envoyées : num = 6, nom = Zied, date = 2025-09-01, ajouter = ok

```
Veuillez remplir tous les champs.
Total : 4 serveur(s)
```

→ Une clé primaire en double fait échouer l'`INSERT` : `mysqli_query` renvoie `false` et `affected_rows` vaut -1 (grâce à `mysqli_report(MYSQLI_REPORT_OFF)` de `config.php`). Le nom est inséré tel quel : l'injection SQL est hors programme, mais à garder en tête.

Exercice 8.5 – Modifier ou supprimer une commande (radio + UPDATE / DELETE)

Écrire `gestion.php` (POST) : champ `num` (numéro de commande), champ `prix` (nouveau prix), boutons radio `action` (`modifier` / `supprimer`) et bouton `valider`. Pour `modifier` : exécuter l'`UPDATE` du prix ; `affected_rows` = 1 → « Prix modifié. », 0 → « Aucune modification (numéro inconnu ou même prix). ». Pour `supprimer` : exécuter le `DELETE` ; 1 → « Commande supprimée. », 0 → « Numéro inconnu. ». Si aucune action n'est choisie ou si `num` n'est pas numérique : « Saisie incorrecte. ».

Solution :

Fichier : gestion.php

```php
<?php
require "config.php";
$cnx = mysqli_connect(HOTE, UTIL, MDP, BASE) or die("Connexion impossible");
echo "<div id='resultat'>";
if (isset($_POST["valider"])) {
    if (!isset($_POST["action"]) || !is_numeric($_POST["num"])) {
        echo "<p>Saisie incorrecte.</p>";
    } else {
        $num = (int)$_POST["num"];
        if ($_POST["action"] == "modifier") {
            $prix = (float)$_POST["prix"];
            mysqli_query($cnx, "UPDATE COMMANDE SET prix = $prix WHERE num = $num");
            if (mysqli_affected_rows($cnx) == 1) {
                echo "<p>Prix modifié.</p>";
            } else {
                echo "<p>Aucune modification (numéro inconnu ou même prix).</p>";
            }
        } else {
            mysqli_query($cnx, "DELETE FROM COMMANDE WHERE num = $num");
            if (mysqli_affected_rows($cnx) == 1) {
                echo "<p>Commande supprimée.</p>";
            } else {
                echo "<p>Numéro inconnu.</p>";
            }
        }
    }
}
echo "</div>";
mysqli_close($cnx);
?>
<form method="post">
  N° commande : <input type="text" name="num">
  Nouveau prix : <input type="text" name="prix"><br>
  <input type="radio" name="action" value="modifier"> Modifier
  <input type="radio" name="action" value="supprimer"> Supprimer
  <input type="submit" name="valider" value="Valider">
</form>
```

Cas « modifier la commande 1, prix 7 » – données envoyées : num = 1, prix = 7, action = modifier, valider = ok

```
Prix modifié.
```

Cas « modifier la commande 1 avec le même prix (6) » – données envoyées : num = 1, prix = 6, action = modifier, valider = ok

```
Aucune modification (numéro inconnu ou même prix).
```

Cas « supprimer la commande 12 » – données envoyées : num = 12, prix = , action = supprimer, valider = ok

```
Commande supprimée.
```

Cas « supprimer la commande 99 » – données envoyées : num = 99, prix = , action = supprimer, valider = ok

```
Numéro inconnu.
```

Exercice 8.6 – Ancienneté des serveurs (dates et base de données)

Écrire `anciennete.php` : pour chaque serveur (`mysqli_fetch_array`), afficher « Nom : N an(s) » où N est l'ancienneté en **années entières** au **1er janvier 2027**, calculée en PHP à partir de `date_embauche` (extraire l'année avec `substr`), puis « (plus de 5 ans) » si N > 5, sinon « (5 ans ou moins) ».

Solution :

Fichier : anciennete.php

```php
<?php
require "config.php";
$cnx = mysqli_connect(HOTE, UTIL, MDP, BASE) or die("Connexion impossible");
$res = mysqli_query($cnx, "SELECT nom, date_embauche FROM SERVEUR");
while ($l = mysqli_fetch_array($res)) {
    $embauche = $l["date_embauche"];
    $anc = 2027 - substr($embauche, 0, 4);
    if (substr($embauche, 5) > "01-01") {
        $anc--;
    }
    echo $l["nom"] . " : $anc an(s) ";
    echo ($anc > 5) ? "(plus de 5 ans)" : "(5 ans ou moins)";
    echo "<br>";
}
mysqli_close($cnx);
?>
```

Affichage obtenu

```
Sami : 5 an(s) (5 ans ou moins)
Rim : 3 an(s) (5 ans ou moins)
Karim : 7 an(s) (plus de 5 ans)
Nour : 2 an(s) (5 ans ou moins)
```

→ Ancienneté au 1er janvier 2027 = 2027 − année d'embauche, moins 1 si l'embauche est postérieure au 1er janvier. Sami (embauché le 01/01/2022) a **exactement** 5 ans : « 5 ans ou moins ».

Exercice 8.7 – Commandes d'un mois (liste déroulante et dates SQL)

Écrire `commandes_mois.php` (POST) : liste déroulante `mois` (noms français générés par une boucle, `value` de 1 à 12), liste `annee` (2026, 2027) et bouton `voir`. Afficher les commandes dont `date_cmd` est dans le mois choisi (`MONTH(date_cmd) = $mois AND YEAR(date_cmd) = $annee`) dans un tableau (plat, quantité, date au format `jj/mm/aaaa`) puis le chiffre d'affaires du mois (`prix × qte`). S'il n'y a aucune commande : « Aucune commande ce mois-ci. » (`mysqli_num_rows`).

Solution :

Fichier : commandes_mois.php

```
<?php
require "config.php";
$cnx = mysqli_connect(HOTE, UTIL, MDP, BASE) or die("Connexion impossible");
mysqli_set_charset($cnx, "utf8mb4");
$noms = ["", "Janvier", "Février", "Mars", "Avril", "Mai", "Juin",
         "Juillet", "Août", "Septembre", "Octobre", "Novembre", "Décembre"];
echo "<div id='resultat'>";
if (isset($_POST["voir"])) {
    $mois = (int)$_POST["mois"];
    $annee = (int)$_POST["annee"];
    $res = mysqli_query($cnx, "SELECT plat, prix, qte, date_cmd FROM COMMANDE
                               WHERE MONTH(date_cmd) = $mois AND YEAR(date_cmd) = $annee");
    if (mysqli_num_rows($res) == 0) {
        echo "<p>Aucune commande ce mois-ci.</p>";
    } else {
        $ca = 0;
        echo "<table border='1'>";
        while ($l = mysqli_fetch_array($res)) {
            echo "<tr><td>" . $l["plat"] . "</td><td>" . $l["qte"] . "</td>";
            echo "<td>" . date("d/m/Y", strtotime($l["date_cmd"])) . "</td></tr>";
            $ca += $l["prix"] * $l["qte"];
        }
        echo "</table><p>Chiffre d'affaires de " . $noms[$mois] . " $annee : $ca DT</p>";
    }
}
echo "</div>";
?>
<form method="post">
  <select name="mois">
<?php
for ($m = 1; $m <= 12; $m++) {
    echo "    <option value='$m'>" . $noms[$m] . "</option>\n";
}
?>
  </select>
  <select name="annee"><option>2026</option><option>2027</option></select>
  <input type="submit" name="voir" value="Voir">
</form>
```

Cas « mars 2026 » – données envoyées : mois = 3, annee = 2026, voir = ok

```
Salade tunisienne | 2 | 15/03/2026
Brik à l'oeuf | 5 | 15/03/2026
Couscous à l'agneau | 3 | 22/03/2026
Poisson grillé | 2 | 28/03/2026
Chiffre d'affaires de Mars 2026 : 154 DT
```

Cas « janvier 2026 » – données envoyées : mois = 1, annee = 2026, voir = ok

```
Aucune commande ce mois-ci.
```

→ Vérification avec la série 8 : le chiffre d'affaires de mars 2026 est 154 DT (12 + 20 + 66 + 56).

Enseignant : Aymen Essouyah – Lycée Rafèha – 4ème SI